Prelim Laboratory Exam: Software Construction & Collaboration

---

1. Implemented features summary

The primary deliverable for Sprint 1 is a robust, responsive Task Management System developed using PySide6 and ensures a **scalable Layered Architecture**.

The core features include **full CRUD capabilities wherein users can Create, Read, Update, and Delete tasks** which simplifies workflow management with prioritized lists and instant "Mark as Done" functionality.

To guarantee system integrity and user data security, the application **incorporates rigorous input validation and parameterized database queries**, effectively preventing common vulnerabilities like SQL Injection while maintaining high performance with a local SQLite backend.

2. Coding Standards Applied

The project follows consistent naming conventions for variables and functions. Proper indentation and formatting were maintained to ensure readability. Comments were added where necessary to explain important logic. Modular programming principles were applied to separate concerns and improve maintainability.

3. Git Contribution Summary

Git was used to manage version control throughout development. Commits were made regularly with meaningful messages describing the changes. Branching was utilized when implementing new features or fixes to keep the main branch stable. The repository reflects continuous progress and organized updates.

Commits on Feb 15, 2026

[EP01-BS01] Creation of Base Files (#17) radzzxc committed 4 days ago		2ad7e67
[EP01-BS01] Creation of Base Files radzzxc committed 4 days ago		f67696c
Revise commit message format and merge rules ARNMA03 authored 5 days ago	Verified	1602226
Add commit message format section to GIT_WORKFLOW.md ARNMA03 authored 5 days ago	Verified	50288c4

Commits on Feb 14, 2026

Revise merge rules formatting in GIT_WORKFLOW.md ARNMA03 authored 5 days ago	Verified	78067e0
Add merge rules section to GIT_WORKFLOW.md ARNMA03 authored 5 days ago	Verified	f723340
Add system architecture diagram jerstin-mane authored 5 days ago	Verified	3aca0c9
Establish project architecture folder structure jhoanna-delapaz authored 5 days ago	Verified	37a7974
Establish project architecture folder structure jhoanna-delapaz authored 5 days ago	Verified	ac8e8ff
Establish project architecture folder structure jhoanna-delapaz authored 5 days ago	Verified	ba3d33c
Create sample file jhoanna-delapaz authored 5 days ago	Verified	7ff7bae
Delete docs/architecture/sample file jhoanna-delapaz authored 5 days ago	Verified	f8e5259

Commits on Feb 19, 2026

Update DOCUMENTATION.MD fithyrch authored 18 minutes ago	Verified	5c82958
Update DOCUMENTATION.MD fithyrch authored 42 minutes ago	Verified	c77e070
Update DOCUMENTATION.MD fithyrch authored 44 minutes ago	Verified	3765ce9
[EP01-FT03] Creation of Checkbox for Task radzzxc committed 3 hours ago		877408f
[EP01-FT03] Creation of Checkbox for Task radzzxc committed yesterday		b54d095

Commits on Feb 18, 2026

[EP01-FT01] Created Add New Task radzzxc committed yesterday		7c6d568
Merge pull request #19 from jhoanna-delapaz/feature/EP01-BS01-base-files ARNMA03 authored yesterday	Verified	2128205
Update QA documentation for Austria initialization ARNMA03 authored yesterday	Verified	baa4881
Update Sprint Summary section in documentation ARNMA03 authored yesterday	Verified	374f2c9
Revise Sprint Summary with structured reflection ARNMA03 authored yesterday	Verified	d9875fd
Revise Sprint Summary format in documentation ARNMA03 authored yesterday	Verified	75e6e1e
Update DOCUMENTATION.MD jerstin-mane authored yesterday	Verified	9f6fc40

4. Security Considerations (Already posted in develop branch)

SQL Injection Prevention #1

RISK IDENTIFICATION

Where Risk Exists: src/business/task_manager.py (Database Interaction Layer)

The application interacts with a local SQLite database to perform Create, Read, Update, and Delete (CRUD) operations. If user inputs (like "Task Title" or "Description") were directly concatenated into SQL strings using f-strings or .format(), a malicious user could input SQL commands (e.g., specific meta-characters) to manipulate the database structure or delete data (SQL Injection).

IMPLEMENTATION AND MITIGATION

How We Addressed It: We enforced the use of Parameterized Queries provided by the sqlite3 library. Instead of building query strings dynamically, we used placeholder syntax (?).

Code Evidence (src/business/task_manager.py):

```
# SECURE: Using '?' placeholder
sql = ''' INSERT INTO tasks(title, description, status, created_at, due_date, priority)
          VALUES(?,?,?,?,?,?) '''
cur.execute(sql, (task.title, task.description, ...))

# VULNERABLE (Avoided):
# sql = f"INSERT INTO tasks... VALUES('{task.title}', ...)"
```

By using ?, the database driver treats the inputs strictly as data, not executable code, effectively neutralizing SQL injection attacks.

Input Validation & Data Integrity #2

RISK IDENTIFICATION

Where Risk Exists: src/presentation/add_task_dialog.py (User Interface Layer)

Allowing invalid, empty, or extremely long inputs can lead to application instability, database errors (constraints violations), or confusing user experiences. While less severe than code injection in a desktop app, malformed data (like an empty title or whitespace-only strings) compromises the integrity of the data store and the reliability of the application's logic.

IMPLEMENTATION AND MITIGATION

How We Addressed It: We implemented Input Validation at the interface level before data is ever sent to the business logic or database.

Code Evidence (src/presentation/add_task_dialog.py):

```
def validate_and_accept(self):
    # Sanitize: Strip leading/trailing whitespace
    title = self.title_input.text().strip()

    # Validate: Ensure Title is not empty
    if not title:
        QMessageBox.warning(self, "Validation Error", "Title is required!")
        return  # Block submission

    self.accept()
```

- **Sanitization:** .strip() removes accidental whitespace, preventing "blank" tasks that look empty but technically contain spaces.

- **Validation:** Checking if not title ensures that every task has a valid identifier, preventing the creation of "ghost" tasks in the database.

5. Code Review & Reflection

- The codebase is cleanly separated into Presentation (UI), Business Logic, and Data layers. This internal structure makes the application highly maintainable and scalable, enabling you to modify the interface or database independently without breaking the core application logic.

- One area for improvement is improving collaboration through GitHub. Not all changes were discussed before being merged into the main branch. This can sometimes cause confusion or overlapping work. In the future, better communication and review before merging would improve teamwork.

- One lesson we learned is that it is hard to complete tasks when group members are not on the same page. Lack of clear communication can cause confusion and delays. We realized that regular updates and clear roles are important for better teamwork. Good communication makes the implementation process smoother and more organized.

The Sprint 1 Code Review confirms that the EFFICIO application is stable and tailored to its core functional requirements. Testing verified a smooth database initialization and application launch, ensuring a robust technical foundation (Austria). Functional testing validated that all key task management features (Add, Edit, Delete, Mark as Done) operate correctly without major failures (Villafranca). While the current implementation is solid, user feedback indicates a need

for a "Reminder Feature" to alert users of upcoming deadlines, which will be the primary focus for enhancement in Sprint 2.

Group Members:

- Austria, Arden Roland Nicholai M.
- De La Paz, Jhoanna Alexandra C.
- Emperador, Radge Michael A.
- Mane, Jerstin M.
- Villafranca, Richardson M.